TikuKits ML: k-Nearest Neighbors for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 14, 2026

Outline

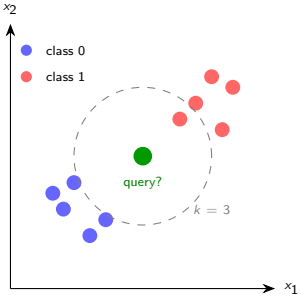
Why k-NN on MCUs?

Embedded Use Cases

- **No training phase** — just store examples
- **Adaptive** — add new samples on-the-fly
- **Multi-class** — naturally handles any number of classes
- **Non-parametric** — no assumptions about data distribution

Constraints

- No FPU (16-bit MSP430)
- No heap allocation
- Limited RAM (2–8 KB)
- Ring buffer keeps memory bounded



Design Goal

Instance-based classification with **bounded memory** via ring buffer, squared Euclidean distance (no `sqrt`), and majority voting among k nearest neighbors.

k-NN Classification

Distance metric: Squared Euclidean

$$d(\mathbf{a}, \mathbf{b}) = \sum_{j=1}^{n} (a_j - b_j)^2$$

No `sqrt()` needed — ordering is preserved.

Prediction:

1. Compute distance from query to all stored samples
2. Select k samples with smallest distance
3. Majority vote among their labels
4. Return the most frequent class

Complexity: $O(N \times F)$ per query, where N = stored samples, F = features.

Ring Buffer Storage

When the sample buffer is full, new samples **overwrite** the oldest. This creates a sliding-window effect — the classifier adapts to recent data.

Overflow Safety

Differences are cast to `int64_t` before squaring: $2^{31} \times 2^{31} = 2^{62} < 2^{63}$. Accumulation over 8 features stays within `int64_t`.

The tiku_kits_ml_knn Structure

ml/classification/tiku_kits_ml_knn.h

```c
struct tiku_kits_ml_knn {
    tiku_kits_ml_elem_t
        data[TIKU_KITS_ML_KNN_MAX_SAMPLES]
            [TIKU_KITS_ML_KNN_MAX_FEATURES];
    uint8_t
        labels[TIKU_KITS_ML_KNN_MAX_SAMPLES];
    uint16_t n_samples;
    uint16_t write_idx;
    uint8_t  n_features;
    uint8_t  k;
};
```

Memory footprint:

$32 \times 8 \times 4 + 32 + 2 + 2 + 1 + 1 = 1062$ bytes.

All statically allocated.

Configuration macros

```c
/* Max features per sample */
#ifndef TIKU_KITS_ML_KNN_MAX_FEATURES
#define TIKU_KITS_ML_KNN_MAX_FEATURES 8
#endif

/* Max stored samples */
#ifndef TIKU_KITS_ML_KNN_MAX_SAMPLES
#define TIKU_KITS_ML_KNN_MAX_SAMPLES 32
#endif

/* Max output classes */
#ifndef TIKU_KITS_ML_KNN_MAX_CLASSES
#define TIKU_KITS_ML_KNN_MAX_CLASSES 8
#endif

/* Default k */
#ifndef TIKU_KITS_ML_KNN_DEFAULT_K
#define TIKU_KITS_ML_KNN_DEFAULT_K 3
#endif
```

Complete API

Function	Description
`tiku_kits_ml_knn_init(knn, nf, k)`	Initialize with `nf` features, k neighbors
`tiku_kits_ml_knn_reset(knn)`	Clear all stored samples
`tiku_kits_ml_knn_add(knn, x, label)`	Add sample (ring buffer, overwrites oldest when full)
`tiku_kits_ml_knn_predict(knn, x, &result)`	Classify by majority vote of k nearest
`tiku_kits_ml_knn_set_k(knn, k)`	Change k at runtime
`tiku_kits_ml_knn_get_k(knn)`	Get current k value
`tiku_kits_ml_knn_count(knn)`	Number of stored samples

Minimum Data Requirement

`predict()` requires at least k stored samples. Returns `ERR_SIZE` if fewer than k samples are available. Labels must be in range $[0, \text{MAX_CLASSES})$.

Example 1: Temperature Zone Classification

examples/example_ml_knn.c — 3 zones with k=3

```c
struct tiku_kits_ml_knn knn;
uint8_t result;

tiku_kits_ml_knn_init(&knn, 1, 3);   /* 1 feature, k=3 */

/* Add calibration samples: cold(0), normal(1), hot(2) */
int32_t cold[] = {100, 120, 150, 170, 180};
int32_t norm[] = {220, 240, 250, 260, 280};
int32_t hot[]  = {320, 340, 350, 360, 380};

uint8_t i;
for (i = 0; i < 5; i++) {
    tiku_kits_ml_knn_add(&knn, &cold[i], 0);
    tiku_kits_ml_knn_add(&knn, &norm[i], 1);
    tiku_kits_ml_knn_add(&knn, &hot[i],  2);
}

/* Classify new readings */
int32_t query1 = 140;
tiku_kits_ml_knn_predict(&knn, &query1, &result);   /* result = 0 (cold) */

int32_t query2 = 330;
tiku_kits_ml_knn_predict(&knn, &query2, &result);   /* result = 2 (hot) */
```

Example 2: Gesture Recognition

2-D accelerometer: tap/shake/tilt

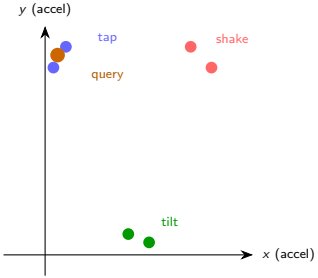
```c
struct tiku_kits_ml_knn knn;
uint8_t result;

tiku_kits_ml_knn_init(&knn, 2, 3);

/* Tap: small X, large Y spike */
tiku_kits_ml_elem_t tap1[] = {5, 50};
tiku_kits_ml_elem_t tap2[] = {-3, 55};
tiku_kits_ml_knn_add(&knn, tap1, 0);
tiku_kits_ml_knn_add(&knn, tap2, 0);

/* Shake: large X, large Y */
tiku_kits_ml_elem_t shk1[] = {40, 45};
tiku_kits_ml_elem_t shk2[] = {-35, 50};
tiku_kits_ml_knn_add(&knn, shk1, 1);
tiku_kits_ml_knn_add(&knn, shk2, 1);

/* Tilt: moderate X, small Y */
tiku_kits_ml_elem_t tlt1[] = {20, 5};
tiku_kits_ml_elem_t tlt2[] = {25, -3};
tiku_kits_ml_knn_add(&knn, tlt1, 2);
tiku_kits_ml_knn_add(&knn, tlt2, 2);
```



Dynamic k

Change k at runtime with `set_k()`:

- Small k (1–3): noisy but responsive
- Large k (5–7): smoother, slower

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_knn_init`	Proper initialization, parameter validation
2	`test_kits_ml_knn_simple`	1-D classification with 3 classes
3	`test_kits_ml_knn_two_features`	2-D classification
4	`test_kits_ml_knn_k1`	Nearest-neighbor ($k = 1$) classification
5	`test_kits_ml_knn_change_k`	Runtime k change via `set_k()`
6	`test_kits_ml_knn_overflow`	Ring buffer overwrites oldest samples
7	`test_kits_ml_knn_negative`	Negative feature values handled correctly
8	`test_kits_ml_knn_reset`	Reset clears all stored samples
9	`test_kits_ml_knn_null_inputs`	All functions reject NULL with ERR_NULL

Summary

What We Built

- Instance-based k-NN classifier
- Ring buffer for bounded memory
- Squared Euclidean distance (no `sqrt`)
- Majority voting with tie-breaking
- $\sim$1 KB state (default 32 samples $\times$ 8 features)

Key Design Choices

- `int64_t` distance accumulation
- Ring buffer for sliding-window adaptation
- Runtime-adjustable k value
- Up to 8 classes supported

Use Cases

- Temperature zone classification
- Gesture recognition (accelerometer)
- Adaptive monitoring with new samples
- Multi-class sensor classification

Files

Header:	`ml/classification/tiku_kits_ml_knn.h`
Source:	`ml/classification/tiku_kits_ml_knn.c`
Tests:	`tests/ml/test_kits_ml_knn.c`
Example:	`examples/example_ml_knn.c`

Thank You

TikuKits ML: k-Nearest Neighbors

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/classification/tiku_kits_ml_knn.h`
Source:	`tikukits/ml/classification/tiku_kits_ml_knn.c`
Tests:	`tikukits/tests/ml/test_kits_ml_knn.c`
Examples:	`tikukits/examples/example_ml_knn.c`